

Data Safety and Recovery

Summary of File System/File Type – Based Recovery

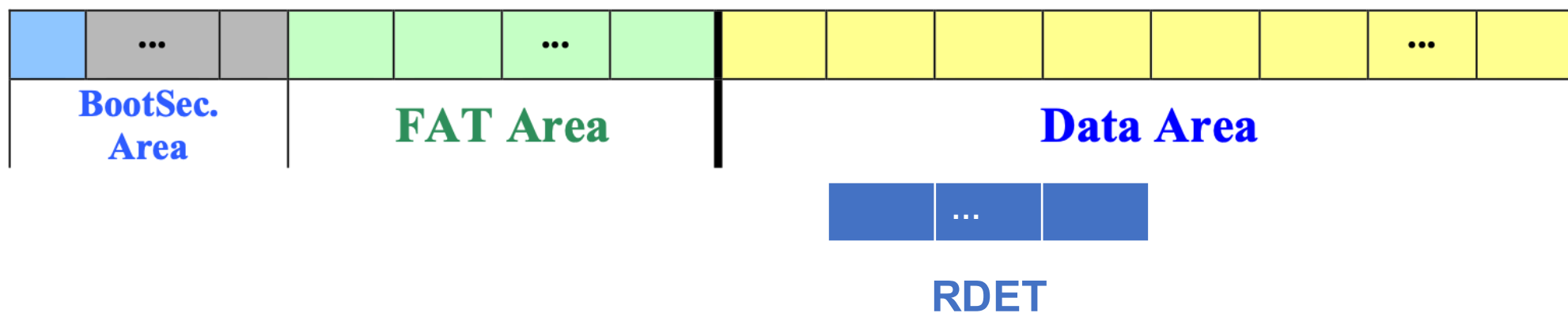


Contents

- FAT32
- ExFAT
- File recovery: GIF, WAV, JPG, PDF, DOCX, MP3

FAT32

FAT32 – Volume Structure



Retrieve files, folders:



FAT32 – Boot sector

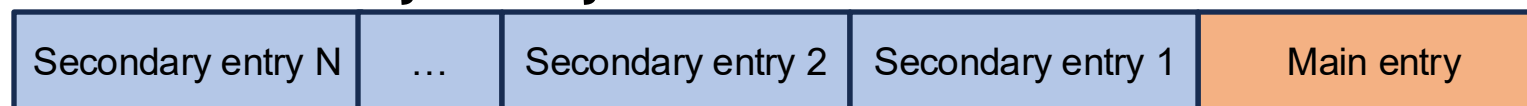
Offset	# of Bytes	Description
B	2	Bytes per Sector, usually 512
D	1	S_C : Sectors per cluster
E	2	S_B : Number of sectors of Boot sector area
10h	1	N_F : number of FAT, usually 2
20h	4	S_V : Volume size
24h	4	S_F : Size of each FAT
2C	4	First cluster of RDET
30h	2	Sector that contains the additional information (about free cluster), usually 1
32h	2	Backup sector of Boot Sector
52	8	FAT type, "FAT32 "

FAT₃₂ - FAT

Value	Status	Description
0	FREE	Free cluster
0x0FFFFFFF7	BAD	Bad cluster
0x0FFFFFFF	EOF	Last cluster of a file/folder
2 → 0x0FFFFFFEF	USED	Cluster belongs to a file/folder. Value is equal to the next cluster index

FAT32 – Directory Entry

- Main and secondary entry order



Entry phụ

Offset	# of Bytes	Data type	Meaning
0	1	Unsigned Integer	Entry order (start from 1)
1	10	Unicode string	5 Unicode characters
B	1	Unsigned Integer	Signature – 0x0F
C	1	Unsigned Integer	Reversed (always 0)
D	1	Unsigned Integer	Checksum
E	12	Unicode string	6 next Unicode characters
1A	2	Array of bytes	Reversed (always 0)
1C	4	Unicode string	2 next Unicode characters

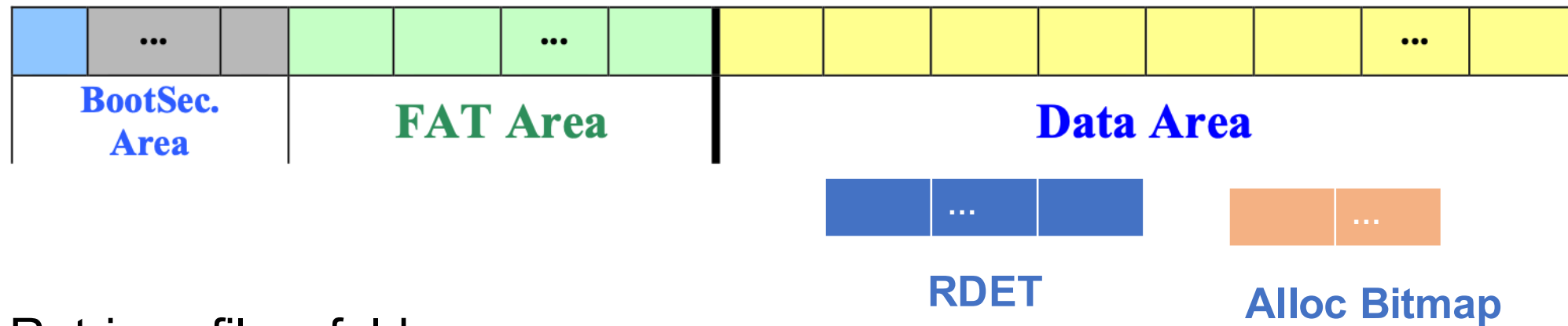
FAT32 – Directory Entry

o Main entry

Offset	# of Bytes	Data type	Meaning
0	8	ASCII string	Short name - ASCII
8	3	ASCII string	Extension - ASCII
B	1	Array of bits	Attribute (0.0.A.D.V.S.H.R)
C	1	Array of bytes	Reversed
D	3	Unsigned interger	Creation time (ms: 7b; s: 6b; m: 6b; h: 5b)
10h	2	Unsigned interger	Creation date (day: 5b; month: 4b; year – 1980: 7b)
12h	2	Unsigned interger	Recent access date
14h	2	Unsigned interger	First cluster – High Word (2 bytes)
16h	2	Unsigned interger	Modification time (s/2: 5b; m: 6b; h: 5b)
18h	2	Unsigned interger	Recent update date
1A	2	Unsigned interger	First cluster – Low Word
1C	4	Unsigned interger	File size (byte)

ExFAT

ExFAT – Volume Structure



Retrieve files, folders



ExFAT – Boot sector

Offset	# of Byte	Meaning
3	8	String “EXFAT”
40h	8	Start position of partition (sector)
48h	8	Volume size (sector)
50h	4	Start position of FAT (sector)
54h	4	FAT size (sector)
58h	4	Start position of Cluster Heap, means data area
5C	4	Number of cluster (maximum $2^{32} - 11$)
60h	2	RDET position
6C	2	Byte per sector (exponent to the base 2). Value range is 9 - 12
6D	1	Sector per cluster (exponent to the base 2). Value range is 20 – 25.
6E	1	Number of FAT

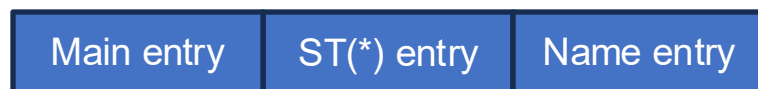
ExFAT - FAT

- In exFAT, FAT isn't used to monitor the cluster allocation; **Allocation Bitmap** is used for that purpose instead

FAT entry value	Status	Description
0x00000000	-	Not mean the free cluster, it is an undefined value
0x00000002 - 0xFFFFFFFF6		Value range
0xFFFFFFFF7	BAD	Bad cluster
0xFFFFFFFF8	-	media descriptor. Giá trị của phần tử đầu FAT
0xFFFFFFFFF	EOF	Last cluster

ExFAT – Directory Entry

○ Entry structure



(*) ST: Stream Extension



Offset	Số byte	Diễn giải
0	1	0x85
1	1	Số entry theo sau (giá trị từ 2 – 18)
4	2	Attributes (_A.D._S.H.R)
8	4	Creation date
C	4	Recent modification date
10h	4	Recent access date
14h	1	Milisecond of creation date
15h	1	Milisecond of recent modification date
16h	1	Timezone of creation date
17h	1	Timezone of recent modification date
18h	1	Timezone of recent access date

ExFAT – Directory Entry

ST entry

→

Offset	Số byte	Diễn giải
0	1	Always 0xC0, signature for Stream Extension.
1	1	NoFatChain flag (bit 1). If bit value = 1, unfragmented data, FAT is not used.
3	1	Name length (number of characters).
4	2	Checksum of name. Names are compared easier
8	8	Actual size of file.
14h	4	First cluster
18h	8	File size on disk (allocation).

ExFAT – Directory Entry

Name entry	Offset	Số byte	Diễn giải
	0	1	Entry type (0xC1)
	2	30	Unicode characters

File Recovery: GIF, WAV, JPG, PDF, DOCX, MP3

File carving Technique

- Recover file based on the structure of raw data content
- *Condition:* File data has not been overwritten on the clusters
- *Method:* Scan the entire raw data space to search for identifying signatures (*Data Carving*)
 - *Header/Magic Number:* A fixed sequence of bytes at the beginning of a file used to identify the file type
 - *Footer/EOF:* A specific sequence of bytes that marks the end of the data stream
 - *File Size:* For files that lack a distinct footer (such as BMP), carving tools will read the size information stored directly within the *Header* to determine how much data needs to be extracted.

File carving Technique

o Basic processes

- *Scanning*: The tool scans the raw data on the disk byte-by-byte (bypassing the system area if the file system information is known).
- *Identification*: Search for *Headers* that match the file signature database
- *Extraction*:
 - If a Footer exists: Extract the data from the *Header* through to the *Footer*.
 - If no Footer exists: Use the file size stored in the *Header* to carve the data
 - For plain text files (such as CSV): Use structural heuristics (e.g., frequency of commas, newline characters) since they lack fixed signatures.

File carving Technique

o Limitation

- *Fragmented data*: If a file is not stored contiguously across adjacent clusters, “concatenate” the data fragments back together is highly complex and often results in corrupted files after recovery
- *Overwritten data*: If new data has partially overwritten the clusters belonging to the old file, complete recovery is impossible.

File carving Technique

- o Specific information for some file types

File type	Header	Footer	Description
GIF	GIF87a hoặc GIF89a	0x3B	
WAV	“RIFF”, file size (4 byte), “WAVE”	(none)	Base on the file size to extract data
PDF	%PDF-	%%EOF	
JPG	FFD8	FFD9	
DOCX	504B0304	504B0506	
MP3	FFFB	(none)	Determine footer: header of the next file or start with a long series of 00 bytes

Questions and Answers